

Publicar

Envie um ZIP com front e API C#. Ganhe uma URL, um banco próprio e versões com volta automática.

Hospedar outros sistemas no mesmo servidor, sem mexer no código da plataforma

Antes

Uma ferramenta nova exigia mexer em 4 lugares (rota, card, endpoint, banco), ficava presa a Angular + .NET e dependia de um deploy de tudo. Um build quebrado derrubava todas.

Agora

Arrastar os arquivos, clicar em **Publicar** e receber uma URL pública. Cada sistema roda isolado, com versões, logs, variáveis e banco próprios.

Cinco respostas definiram o escopo

PERGUNTA	DECISÃO	CONSEQUÊNCIA
Quem publica?	Só o dono	Lista de e-mails em PUBLICADORES. Limites protegem de bugs, sem moderação
O que se envia?	Front + back	Pacote com api/ e web/
Linguagem do back	C#	A saída de dotnet publish roda na imagem oficial aspnet
Banco?	Sim	SQLite por padrão e Postgres próprio por site, sob demanda
Acesso e URL	Público, sslip.io	meu-app.191-252-177-244.sslip.io:8090, sem DNS nem domínio

01

Como funciona

Da pasta à URL em quatro passos

1

Criar o site

Nome e endereço. A URL aparece antes de confirmar.
Opcional: já criar o banco Postgres.

2

Enviar

ZIP ou pasta arrastada. A pasta é compactada no navegador.

3

Publicar

Backup dos bancos, container novo, health check pelo mesmo caminho do visitante.

4

No ar

Em **1,7 s** no teste. Se não responder, a versão anterior volta sozinha.

A TELA

Tudo num lugar só

Envio com progresso, versões com log, voltar versão com ou sem banco, logs do container, variáveis cifradas, parar, iniciar e excluir.

Publicar

eu@exemplo.com

+ Novo site

Demo da Tela
demo-da-tela

v1 no ar

Recados
recados

v1 no ar

v1 no ar.

Demo da Tela

http://demo-da-tela.127-0-0-1.sslip.io:18080

No ar: v1 · .NET 9.0 · Recados.dll + front

Enviar nova versão

Um ZIP ou uma pasta com `api/` (saída de `dotnet publish -c Release -o api`) e/ou `web/` (HTML/CSS/JS ou o `dist/` do Angular/React). O app C# escuta na porta que vier em `ASPNETCORE_URLS`, usa `ConnectionStrings:Default` (SQLite em `/data`) e responde em `/api`.

Arraste o ZIP ou a pasta aqui

Escolher ZIP

Escolher pasta

Publicar

Versões

v1

No ar

.NET 9.0 · Recados.dll + front · 28.9 MB

25/09 18:01

Fechar log

[18:01:58] Recebido: 14704 KB compactado, 29543 KB extraído (API .NET 9.0 (Recados.dll) + front).

[18:01:58] Publicando v1.

[18:01:58] Subindo container .NET 9.0 (Recados.dll, 128 MB).

[18:02:00] Respondeu em /api/health.

[18:02:00] No ar em 1.7 s: http://demo-da-tela.127-0-0-1.sslip.io:18080

Logs do app

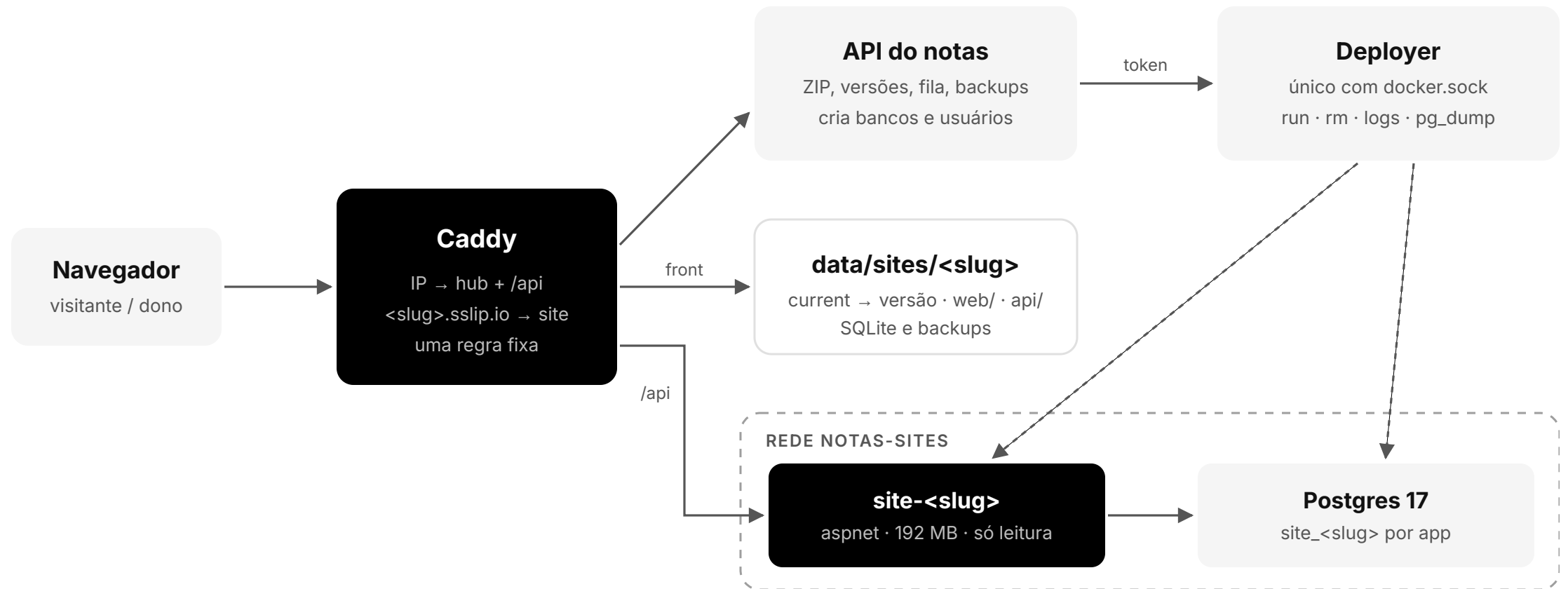
Ver

Variáveis de ambiente

Editar

Chaves de API, senhas, configurações. Guardadas cifradas; use `__` para seções (`Email__Senha`).

Quatro peças, cada uma com um papel



Enviar o C# já compilado, não o código-fonte

OPÇÃO	PROBLEMA	VEREDITO
Compilar na VPS	SDK de ~800 MB e ~1 GB de RAM no build numa máquina de 1.9 GiB	Descartada
Compilar no GitHub Actions	Foge do "arrastar e publicar"	Fase futura
Enviar a saída do <code>dotnet publish</code>	Nenhum: a VPS só executa, na imagem oficial aspnet, com a pasta montada	Escolhida

A versão do .NET (8, 9 ou 10) sai do `*.runtimeconfig.json` do pacote. Não existe imagem por app.

O que um sistema precisa seguir

```
meu-app.zip
├─ api/           dotnet publish
│  └─ MeuApp.dll
│  └─ MeuApp.runtimeconfig.json
├─ web/           front estático
│  └─ index.html
└─ publicar.json opcional
  { "health": "/api/health",
    "memoriaMb": 128 }
```

- 1 Escutar na porta de ASPNETCORE_URLS
- 2 Banco em ConnectionStrings:Postgres OU :Default (SQLite em /data)
- 3 Criar ou migrar o banco na subida
- 4 API sob /api. O resto é o front, com fallback de SPA

02

Um banco para cada app

Banco, usuário e senha criados pela plataforma

Um servidor

Postgres 17 compartilhado, limitado a 256 MB. Usa ~54 MiB ocioso. Um Postgres por app custaria isso vezes N.

Um dono por banco

site_<slug> com senha aleatória de 64 caracteres, guardada cifrada. Sem superusuário e sem acesso aos bancos vizinhos.

Entregue ao app

ConnectionStrings__Postgres e PGHOST, PGUSER, PGPASSWORD... Trocar a senha reinicia o app com a nova.

```
# de dentro da rede dos apps, com a senha do site "outro":  
psql -h postgres -U site_outro -d site_recados → User does not have CONNECT privilege.
```

A TELA

Credenciais à mão, sem expor o banco

Copiar a connection string, trocar a senha, apagar o banco. Para o DBeaver, um túnel SSH até o loopback da VPS.

Banco Postgres

Banco e usuário só deste site no Postgres do servidor. O app recebe `ConnectionStrings:Postgres` e as variáveis `PGHOST`, `PGDATABASE`, `PGUSER` e `PGPASSWORD`.

Host `postgres:5432`

Banco `site_recados`

Usuário `site_recados`

Senha `.....`  

O host `postgres` só é acessível de dentro dos apps publicados. Para abrir no DBeaver, use um túnel SSH (ver DEPLOY.md).

 **Copiar connection string**  **Trocar senha**  **Apagar banco**

Antes de cada troca, o banco é copiado

ANTES

Backup

Cópia do SQLite e pg_dump do Postgres, guardados com a versão.
Se o dump falha, o deploy é cancelado e nada é tocado.

SE FALHAR

Volta sozinho

Sem resposta em 30 s: o app para, o banco é recriado vazio, recebe o pg_restore e a versão anterior sobe.

A QUALQUER MOMENTO

Voltar versão

Qualquer uma das 5 versões guardadas, com ou sem devolver os bancos ao estado de antes.

O banco é recriado em vez de usar `--clean`: assim some até a tabela de uma migration pela metade.

03

Código de terceiros, com limites

Cada risco tem uma barreira

RISCO	BARREIRA
Site publicado lê o login do hub	Subdomínio próprio: outro origin, sem acesso ao token
ZIP malicioso (zip-slip, link, bomba)	Caminhos normalizados, links recusados, teto de 100 MB compactado e 300 MB extraído
API comprometida controla o Docker	Só o Deployer tem o socket: rede interna, token e comandos fixos, sem shell
Bug de um app derruba a VPS	192 MB, meia CPU, 200 processos e no máximo 5 apps .NET
App acessa o notas ou outro app	Rede separada, disco só leitura (exceto /data), usuário 1000, cap-drop ALL

Teste de ponta a ponta com Docker

1,7_s

do envio ao app respondendo pelo Caddy

21_{MiB}

de RAM do app de exemplo ocioso (limite de 128 MB)

54_{MiB}

de RAM do Postgres compartilhado ocioso

773_{KB}

o pacote do exemplo publicado para linux-x64 (14,7 MB sem essa opção)

160_{MiB}

onde parou um app que tentou alocar 1 GB. O hub seguiu no ar

772

testes automatizados: 344 da API, 16 do Deployer, 412 do front

33_s

até a versão anterior voltar sozinha depois de uma versão quebrada

5

versões guardadas por site, com os backups de banco

A v3 foi enviada com a DLL corrompida

18:26:49	pg_dump de site_recados feito antes da troca (2,5 KB)
18:26:49	Container .NET 9 da v3 sobe. Uma "migration pela metade" cria uma tabela
18:27:20	30 s sem resposta: BadImageFormatException no log do container
18:27:20	Banco recriado e pg_restore: a tabela da v3 some e os recados ficam
18:27:22	v1 de volta no ar, com os dados intactos. O site ficou fora do ar só durante a tentativa (~33 s)

Três variáveis no .env e o deploy de sempre

```
# na VPS, em /opt/notas-vps
echo "PUBLICADORES=seu-email@exemplo.com"          >> .env
echo "DEPLOYER_TOKEN=$(openssl rand -hex 32)"       >> .env
echo "POSTGRES_SENHA=$(openssl rand -hex 32)"       >> .env
echo "COMPOSE_PROFILES=publicar"                    >> .env
docker pull mcr.microsoft.com/dotnet/aspnet:9.0
docker compose -f docker-compose.yml -f docker-compose.vps.yml up -d
```

Sem essas variáveis nada muda. O passo a passo e a operação estão no `DEPLOY.md`, e um app completo de exemplo está em `exemplos/publicar/recados`.

O que falta e o que pode vir

Validar na VPS

Subdomínios sslip.io na porta 8090, vistos de fora

Apps voltando sozinhos depois de reiniciar a máquina

Memória real com vários apps ao mesmo tempo

Evoluções possíveis

HTTPS e domínio próprio (certificado wildcard)

Publicar a partir do GitHub, com build no Actions

Deploy sem indisponibilidade: subir a versão nova ao lado antes de trocar

PUBLICAR

Arraste. Publique. Está no ar.

Próximo passo: ligar na VPS e publicar o primeiro sistema de verdade.

Branch `claude/poc-multi-app-hosting-bui7kl`